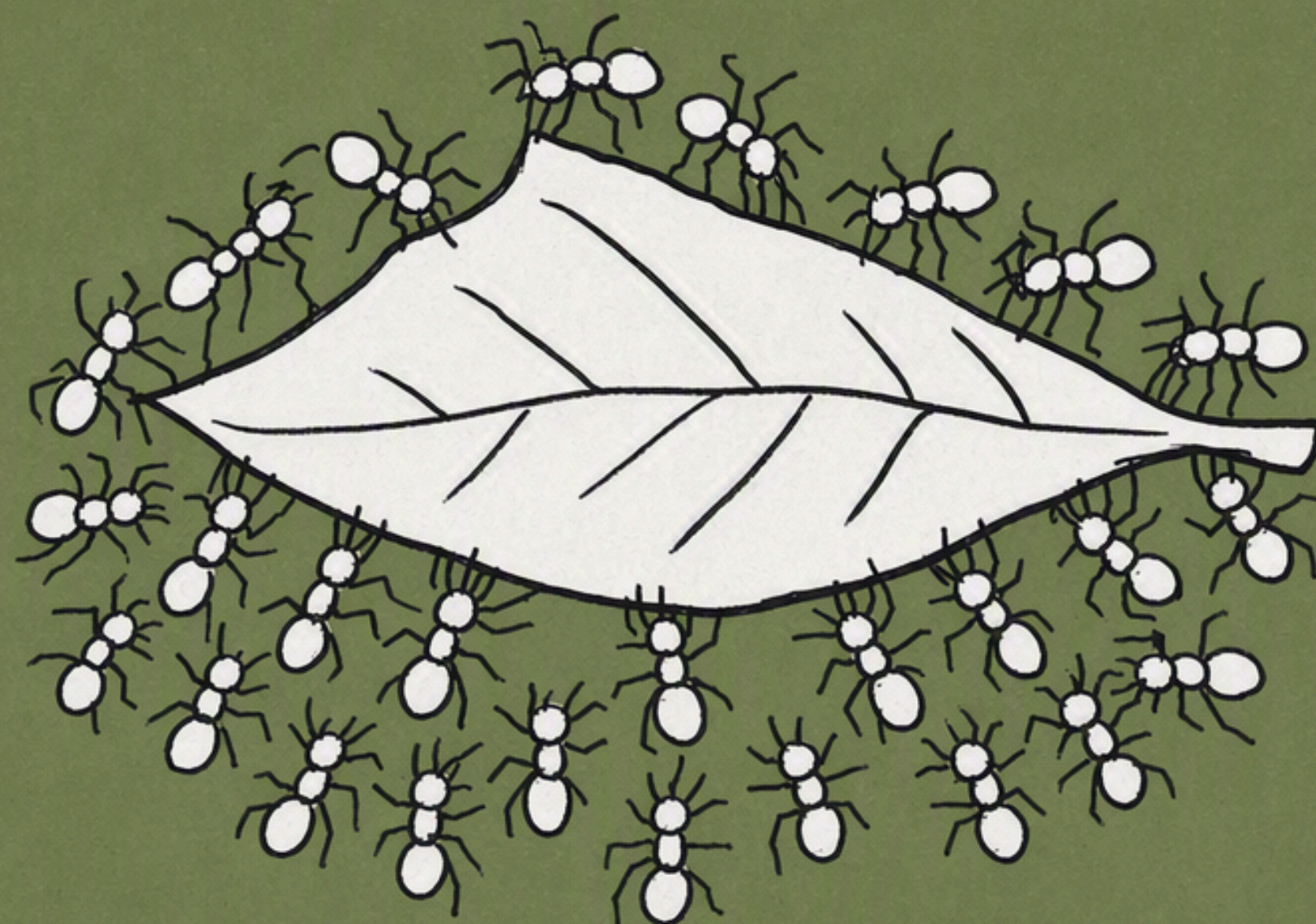




💡 Multi-Agent · 工程实践 📅 2026.8.4 📖 1.5 万字 ⌚ 约 30 分钟

Multi-agent 的新趋势：从 Agent Team 到 Agent Swarm（上）

能同时跑起来的 agent，突然不是个位数了。

今年 2 月我写《Scaling起来 01：让 AI 并行调研 50 个 benchmark》的时候，Kimi K2.5 的 Agent 集群刚上线没多久。我给它派了同一个活——同时去查 50 个 benchmark、每个填一张表。实测下来我给了 0 分：它一个字段都没真去查，把论文里现成的跑分抄了一遍就交差了。

那时候 "agent team" 给我的感受是：概念很性感，产品很难用。

半年过去，agent team 的热度突然最近几个月又起来了。看些时间线

- 1 月 27 日，Kimi K2.5 发布，首次引入 Agent Swarm，官方博客的说法是能自主指挥最多 100 个子智能体。

- 2 月初，Claude Code 放出 Agent Teams 研究预览。官方建议的规模是 3 到 5 个 teammate，文档原话是"三个专注的 teammate，常常胜过五个散的"。
- 2 月 5 日，Cursor 讲他们编排"数千个 agent"协作开发一个真实项目，峰值一小时 1,000 次 commit，连跑一周，累计一千万次工具调用。
- 3 月中，Codex 的 subagents 结束预览正式放开，主 agent 带着 explorer / worker 一起跑。
- 4 月 20 日，K2.6 发布并开源，帮助中心把子智能体上限提到 300 个、单次任务超 4,000 次工具调用。
- 5 月下旬，Anthropic 又在 Claude Code 里上了 Dynamic Workflows，单次运行的 agent 总量上限写的是 1,000 个。
- 5 月 27 日，MiniMax 把自家 Agent 整体升级、起名 Mavis，同时上了 Agent Team，并写了一篇很实在的设计复盘。
- 7 月 20 日，Cursor 第二篇把同一个数推到了一秒 1,000 次 commit。

标题里的 agent team 和 agent swarm 这两个词，不用太较真——它们要解决的是同一件事：把一个任务拆开、同时跑很多个 agent、让效率和效果都得到明显提升。

MiniMax 管自己那套叫 Agent Team，一个 Leader 带几个固定角色的 Worker；Kimi 管自己那套叫 Agent Swarm，官网表示"无需预定义角色或手工设计工作流程"，最多 300 个。Anthropic——它两样都做了，一个产品叫 Agent Teams，官方建议 3 到 5 个；另一个叫 Dynamic Workflows，单次上限 1,000 个。

目前市面上的 agent 大多还是 orchestrator-worker 那一套——Anthropic 的 deep research 是最标准的样本，一个主 agent 负责协调整个过程，把活委派给一批并行运行的专用子 agent。但这半年，明显开始往外走了。

那当单次任务可运行的 agent 数量上来之后呢？这篇分四章往下走，最后再聊点个人感受。

第一章，多 agent 到底有哪些形态。 先把多agent的定义弄清楚：Kimi 按"协作风格"分了五类，LangGraph 按"拓扑"分了五种。

第二章，agent 之间怎么协同。 先看 Evomap 的实验——同一个模型、同一批题，只换组织方式，结果差了 44 个点；再看 MiniMax、Codex、Claude Code 各自的解法。最后是我自己最想搞清楚的一件事：我过去认为并行调研适合多 agent，但写作和代码生成不适合——你让几个 agent 改同一个仓库，不冲突才怪。可基模越来越强，而想上多 agent 的理由又很朴素：我们就是要快啊。10 分钟并行写完一个代码仓库，和 60 分钟串行写完，这不是一回事。那冲突到底怎么解决？这部分看 Cursor。

第三章，怎么把它原生训进模型里。 前面几家都是在 harness 层写代码，Kimi 的论文里讲述了怎么把编排能力直接 RL 进了模型权重。这一章节是偏模型训练层面，模型并不会自发学会并行，它会偷懒退回单 agent 干活，得专门设计一个奖励项把它逼出来。

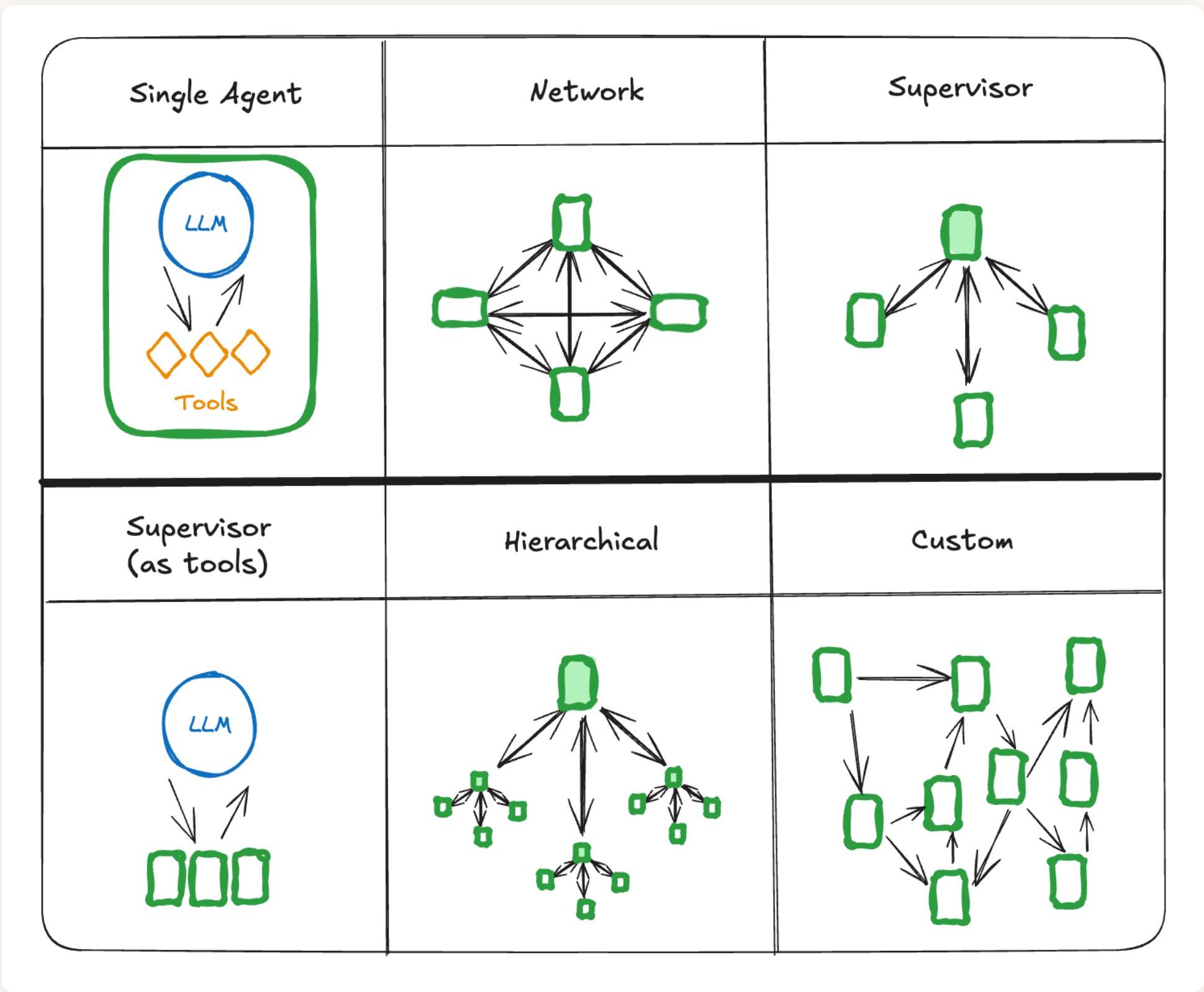
第四章，怎么用它激发智能、找到真正的创新。 前三章都是在用 agent 数量换速度，这一章问的是能不能换智能——当我真想让它做一次头脑风暴，怎么 scaling 出 100 个 agent，再从里面挑出最好的那 1 个，而不是众数的那一个？

第一章 · 多 agent 到底有哪些形态

在看各家怎么做之前，先把"形态"这件事说清楚：多个 agent 凑在一起工作，到底有几种协作方式？有两套分类个人觉得比较直观。

LangGraph 的文档按拓扑分——消息在谁和谁之间流动；Kimi 的科普页按协作风格分——大家各自扮演什么角色。

● LangGraph-按拓扑分的五种连法



LangGraph 文档里的多 agent 架构图。注意左上角的单 agent 不是一种多 agent 架构，是放在那儿做对照的基线；真正的五种是网状、主管式、主管式（把 agent 当工具调）、分层式、自定义

形态	消息怎么流	谁有权决定下一步
单 agent	一个 LLM 拿着一组工具，自己从头做到尾	它自己
网状（Network）	每个 agent 都能跟其他任意一个通信	任何一个 agent 都可以决定下一个叫谁
主管式（Supervisor）	每个 agent 只跟一个主管通信	只有主管
主管式（Subagent as tool）	上一种的特例：每个 agent 被包装成一个工具	只有主管，而且它用"调工具"的方式来决定调谁、传什么参数
分层式（Hierarchical）	主管之上还有主管	各层的主管，逐层往下
自定义（Custom）	每个 agent 只跟一部分 agent 通信	流程里有一部分是写死的，只有某些 agent 有权决定下一步

主管式（Supervisor）和主管式（Subagent as tool）可能有点绕。

Subagent as tool，是指主管不再用自然语言喊"接下来轮到检索 agent 了"，而是像调一个普通函数那样直接发出 `检索agent(关键词="2026年Q1财报")` 这种调用——agent 在主管眼里就是一个有

名字、有参数、有返回值的函数。好处是传了什么参数都是结构化的、程序能校验，不会像自然语言那样传着传着就走样。它跟上一行"主管式"的权力结构完全一样，差别只在主管派活时用的是"说话"还是"调函数"。

这张图把 "谁有权决定下一步"画出来了：网状是人人都有权，主管式是只有主管有权，自定义是一部分写死。而这条线，恰好就是后面几章各家分歧的根源。

● Kimi-常见的五种多智能体架构

Kimi 的这一页是另一个切法，把生产系统里常见的组织方式归成五类。

架构	它回答的问题	怎么组织	原文给的例子
分层式	谁决定下一步	顶层主管拆解目标、把子任务派给下层专家，专家回传结果、主管综合	内容生产流水线：主管拿到一份选题简报，把"查资料"派给一个 agent、"起草"派给第二个、"编辑"派给第三个，最后自己在发布前审一遍整合稿。每个专家只管自己那一段，主管负责让整篇读起来像一个人写的
协作式	谁决定下一步	平级同伴，靠一块共享工作区（大家读写同一份东西），或者一条公共消息通道（谁有发现就往里播一条，其他人都收得到），实时共享工具、数据和中间结果	客服系统：一条投诉进来，一个 agent 做情绪分析、一个查订单历史、第三个起草回复，三者在同一线程里边干边同步。因为发现都汇总在共享工作区，没有任何一个 agent 需要在脑子里装下完整的客户档案
对抗式	成员之间什么关系	内置相反的目标，让它们互相挑战	安全测试：红队 agent 往系统里注入畸形输入、把漏洞串起来打，蓝队 agent 在每个入口冒出来时检测和修补。单个"审核 agent"只朝一个目标努力时容易漏掉的边缘情况，会在这种交锋里被逼出来
异构式	成员长得一样吗	不同能力、不同模型、不同工具的 agent 混编	金融分析流水线：一个轻量的快速分类器高频扫实时行情里的异常，一个大模型写宏观评论和风险判断，第三个专门去调外部 API 取数据。每个 agent 都为自己那摊活用"恰到好处"的模型和工具
基于图	流程写不写出来	agent 和步骤都是图上的节点，每条边定义下一步跑什么	深度研究：先做一轮广撒网的搜索，再分叉出若干子主题并行深挖；初步发现不够充分就绕回去再收集一轮；只有质量过线才进入最终综合

看懂这张表的关键，是认清哪几行在同一个维度上：

- 分层式和协作式是一对，区别就一条：分层式里"下一步谁干"是主管说了算，协作式里是成员自己看着办。对着上面 LangGraph 那张图，前者是主管式，后者是网状。
- 对抗式根本不在这个维度上。它问的是另一码事：这两个 agent 的目标是不是打架的——一个分层式系统里完全可以塞进一对红蓝队。
- 异构式关心的也不是拓扑，是成员配置。这一条既是效果问题，更是成本问题——第二章 Cursor 那笔 8 倍的账，讲的就是它。
- 基于图跟前四类的关系最微妙。前四类讲的是"这群 agent 长什么样"，它讲的是"这套流程有没有被提前画出来"。回头看上面那个深度研究的例子：先搜一轮、再分头深挖几个子主题（这是分支），发现材料不够就绕回去重搜（这是循环），质量够了才往下走综合（这是条件判断）。如果你只能写成"第一步、第二步、第三步"这样一条直线，这三件事一件都表达不了；画成图——节点是步骤，边上标着"什么情况下走这条"——才装得下。

所以它们是可以叠着用的，后面几章出场的每一家都是混搭：

- MiniMax 那套 Leader / Worker / Verifier，同时是分层式（Leader 派活）和对抗式（Verifier 专挑 Worker 的错）。
- Cursor 是分层式，而且层数是跑起来才决定的——上面的 planner 觉得活还能再拆，就现场生一个下级 planner 出来。
- Kimi 的 Agent Swarm 也是分层式，但它连角色表都不预先写，orchestrator 一边跑一边把子 agent 造出来。所以它恰恰不是"基于图"——图压根没被画出来。
- Claude Code 的 Dynamic Workflows 是这里面唯一真正"基于图"的一个：它干脆让 Claude 把整张图写成一段 JavaScript 脚本。
- 第四章还会出现一家叫 Apodex 的 case（陈天桥投的那家，原名 MiroMind），它三样都占——分层、对抗，再加一堆不同职能的异构子 agent。

形态就讲到这儿。但光有形态还不够——真正难的是这些 agent 凑到一起之后，context 怎么传递。

第二章 · agent 之间怎么协同（context engineering）

• Evomap-同一个模型下，三种组织方式的对照实验

Evomap 这家公司（EVOMAP PTE. LTD.，新加坡注册）。它既不做模型也不做 coding harness，做的是 agent 的"经验基础设施"——核心是一套叫 GEP（Genome Evolution Protocol）的协议，把某个 agent 验证过的有效做法打包成可继承的"基因胶囊"，别的 agent 直接拿去用，官网口号是"一个 agent 学会，一百万个继承"。在这之上它也做 agent swarm 平台，下面实验里的 EvoX 就是它自家的蜂群实现。

• 实验一：只换组织方式，能差多少

《From 26% to 71% with the Same Model: How an AI Swarm Wins》(同一个模型，从 26% 到 71%：AI 蜂群是怎么赢的)。设置也很朴素：同一个模型 (Claude Haiku 4.5)、同一批 563 道题、同一套判分规则，只换组织方式。

三种模式的差别是这样的：

- single-context (单上下文)：一个 agent 从头做到尾，563 道题全塞在同一个对话里。做到后面，前面的题早被挤出记忆了。
- Sub-Agent 模式：一个主协调 LLM 先把题分成几摞派给子 agent；子 agent 做完，用自然语言写一份报告交回来；主协调 LLM 读完所有报告，自己整理出最终答案表。
- EvoX swarm：拆到不能再拆——一题一个 agent，各自把答案写到指定位置，最后由一段程序按题号把答案捡起来拼成表，没有任何模型再读一遍、总结一遍。

记住第二种和第三种的分野：**最后那步汇总，是模型干的，还是程序干的。** 后面 32 个点的差距，全在这里。

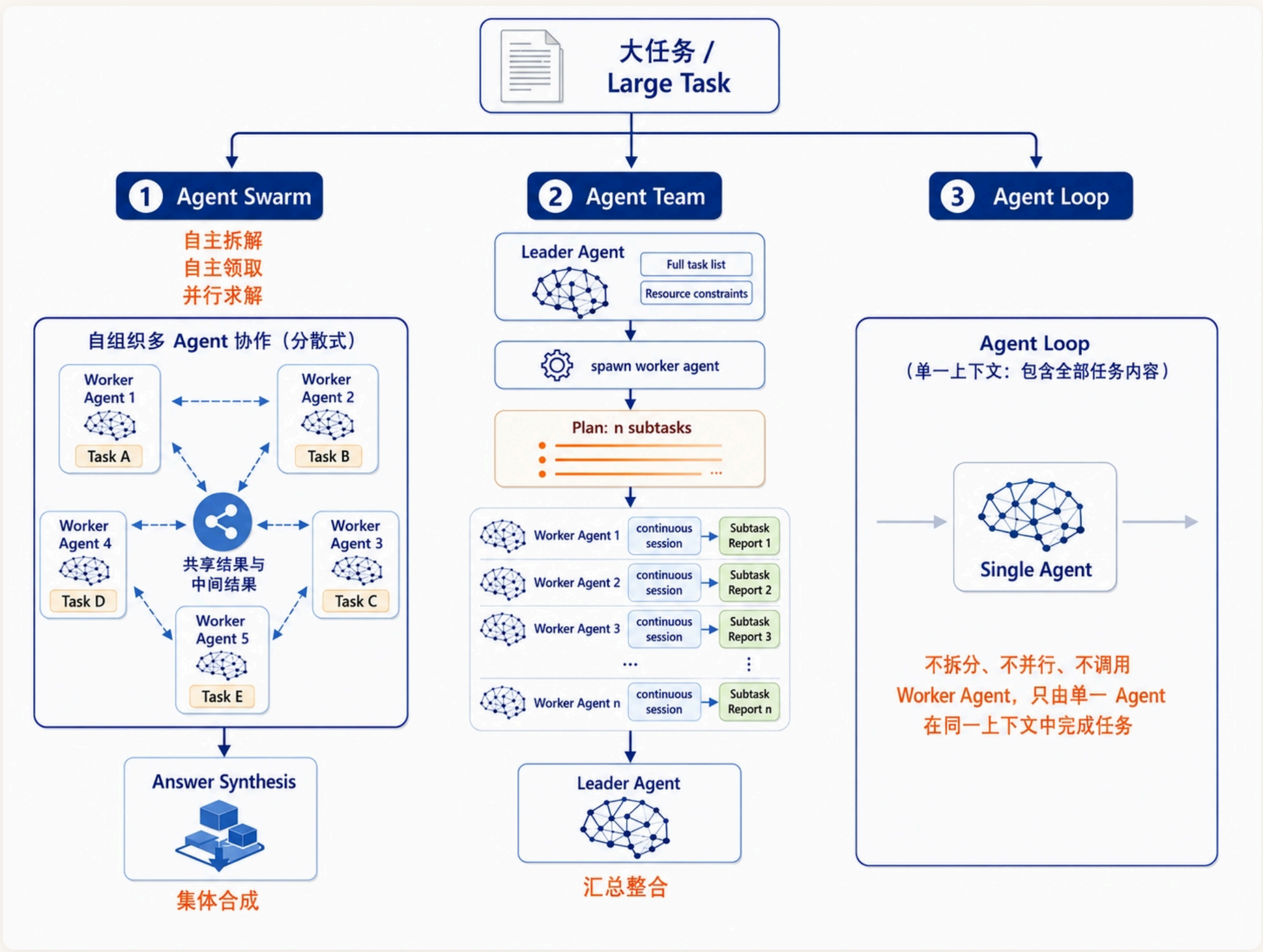


Figure 1: 原文的三种模式叫 EvoX swarm、Sub-Agent mode、single-context mode。左边 EvoX swarm 是把任务尽可能拆成原子单元、每题一个独立 agent；中间 Sub-Agent 模式是一个主协调 LLM 拆活、子 agent 各自汇报、主协调 LLM 再汇总；右边 single-context 就是一个 agent 在同一上下文里做完全部

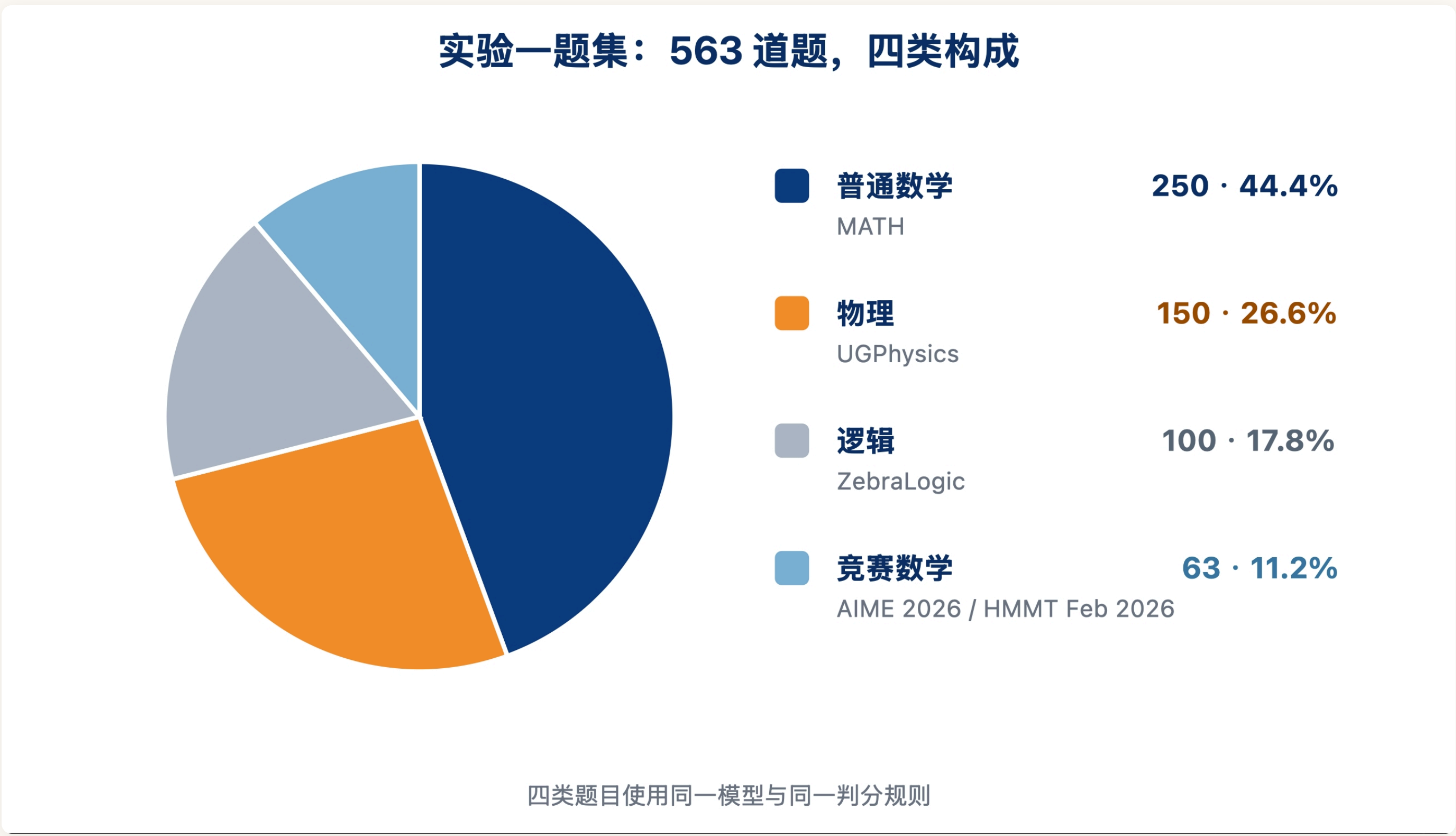


Figure 2: 563 道题的构成——250 道 MATH、150 道 UGPhysics、100 道 ZebraLogic、63 道 AIME / HMMT 竞赛题

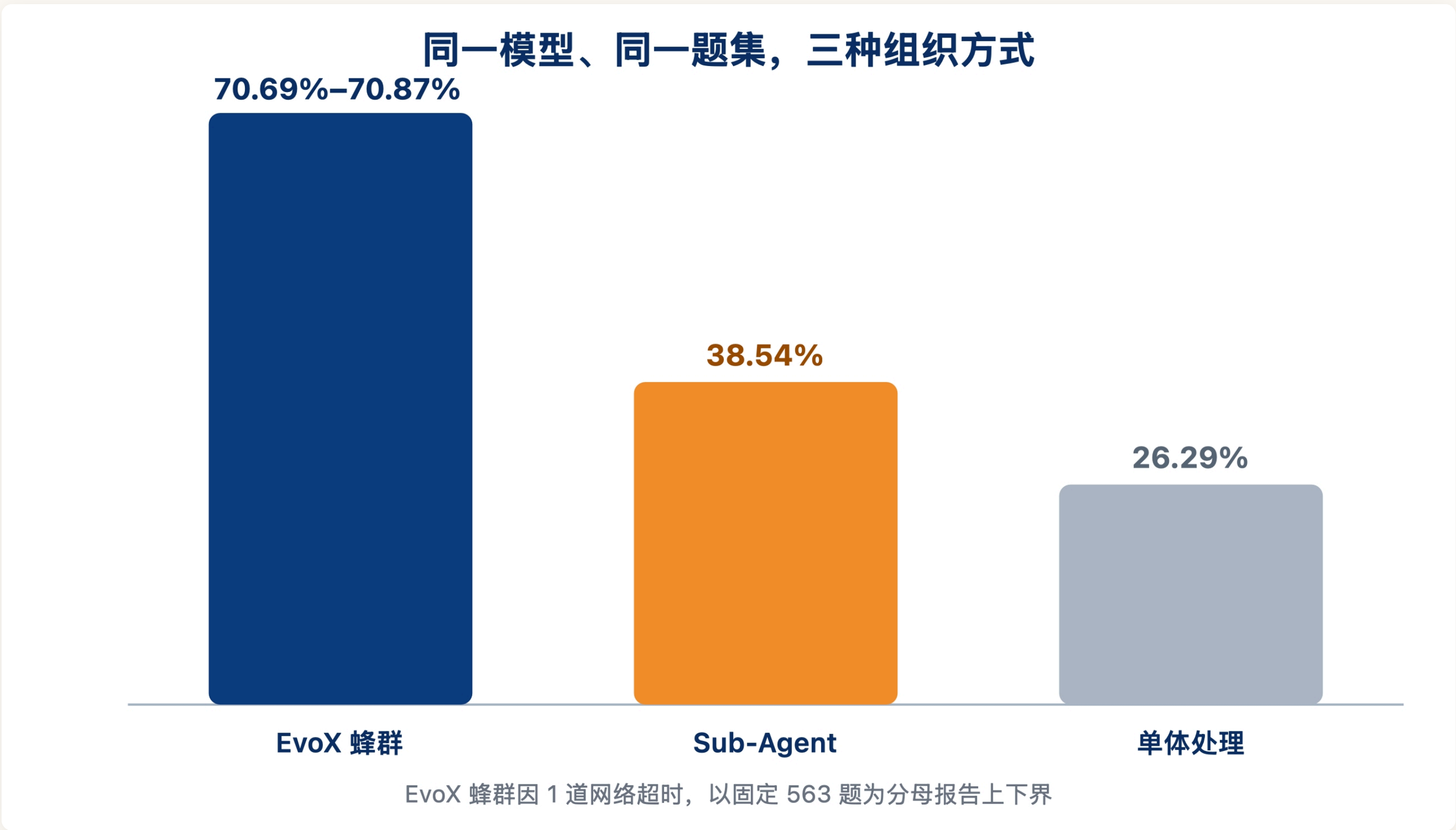


Figure 3: 同模型同题集，single-context 26.29%、Sub-Agent 模式 38.54%、EvoX swarm 70.69%–70.87%

单上下文到 Sub-Agent 涨 12 个点，符合直觉——上下文不打架了。但 Sub-Agent 到 swarm 又涨 32 个点，同一个模型、同一批 563 道题，在 EvoX swarm 里最终答对 398 道，在 Sub-Agent 模式里只答对 217 道。

原文里细拆，这 180 道的差距不是子 agent 算不出来。原文往回追了一层：Sub-Agent 模式的子 agent 在中间环节做对过 373 道——离 398 已经不远了。但这 373 道里，只有 207 道原样活到了最终交付，166 道在传递过程中变成了错的、或者干脆没了，保留率 55.50%。（最终那 217 = 活下来的 207 + 另外 10 道中间答错、在汇总环节反而被扳回来的。）

一句话：不是子 agent 算错了，是子 agent 算对了、然后用自然语言汇报给协调者、协调者再理解一遍、再总结一遍——在这个传递链路上丢掉了。

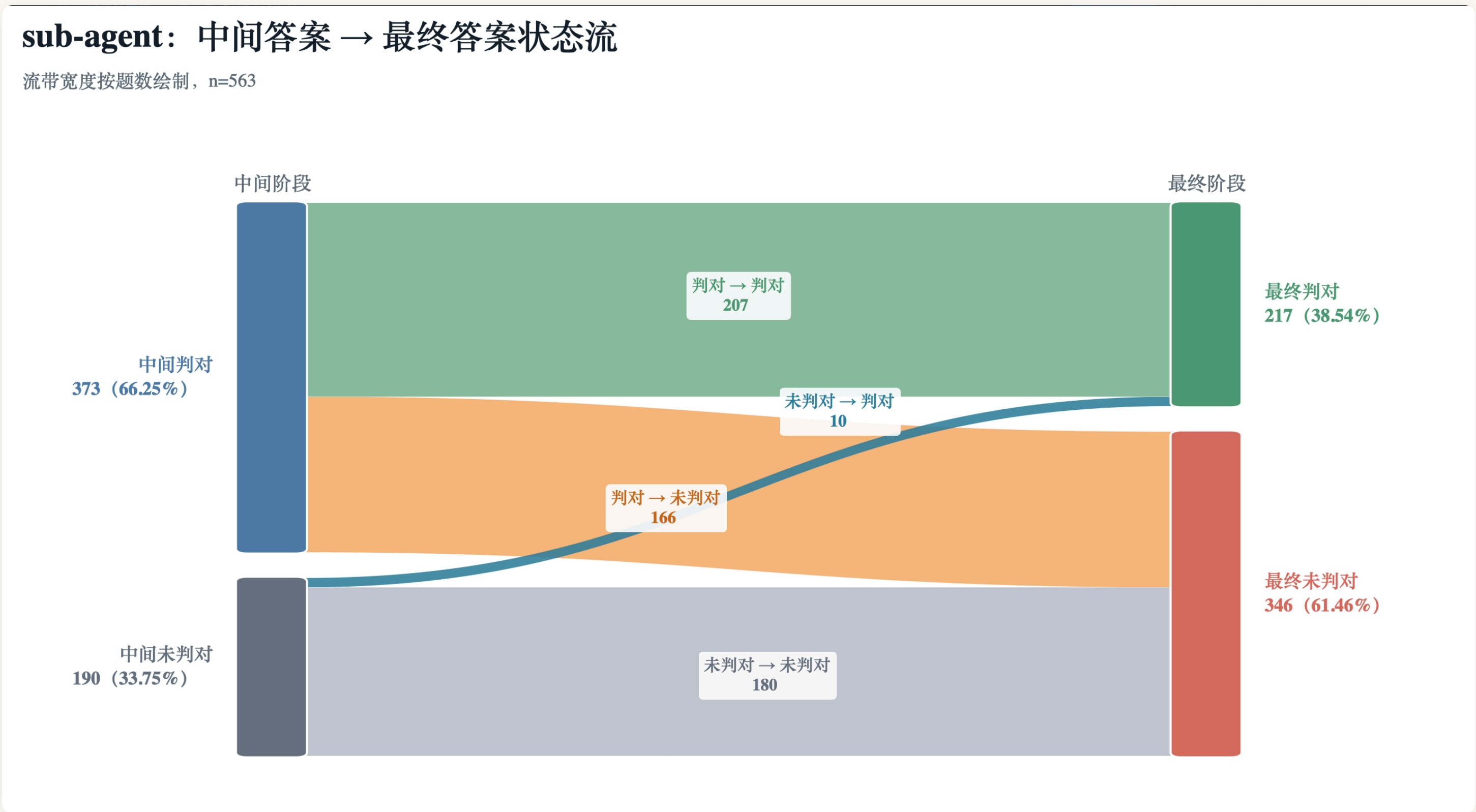


Figure 4: Sub-Agent 流水线里丢掉的正确答案——中间阶段判对 373 道，其中 166 道在传递过程中变成"错误或缺失"，只有 207 道活到最后

EvoX 的解法是：任务拆成边界尽可能清晰的原子单元，每个单元一个隔离的 agent，结果写到指定位置，再由主 agent 自己写的一段程序按题号把答案收上来——注意是"写一段程序去收"，不是"自己读一遍再总结"。没有第二个模型重写结果，也没有一个中心角色决定哪些答案该留。用原文的话说：汇总这一步，不再依赖"重新理解一遍"。

让模型做汇总，等于在系统最后一环塞了一个有损压缩器。

• 实验二：让 agent 自己挑伙伴

它的第二个实验换了个角度。

实验一 Agent Swarm 完整的拆解是脚本保证的——原文自己也承认，现实任务很少像 563 道独立题这么整齐："有些部分互相依赖，有些需要不同专长，有些是执行过程中才产生的，还有些会失败、冲突、重复。"所以剩下一个更难的问题：能不能从"脚本提供完整分工"，走到"agent 自己发现专长、认领任务、挑选伙伴、调整组织"？

实验设置分两段。第一段是让身份先长出来：24 个配置完全相同的 agent 去做题，每做完一道就把"这类题我学到了什么"写进持久记忆；在某一类上积累得越多，下一轮就越可能继续挑那一类。跑满 8 轮之后，系统用它们的实际选择和成绩生成两条身份信息：专长画像和准确率。

以 agent-8 为例，它的起点跟其他 23 个一模一样，但 8 轮里它挑了 5 次物理、2 次普通数学、1 次竞赛数学，最后长成"物理 0.625 / 数学 0.25 / 竞赛数学 0.125"、准确率 0.25 的样子。这个"物理型"身份不是提前写进角色表的，是它自己选出来的一段个人史。

第二段才是组队：24 个 agent 从同一张"圆桌"关系网出发（每个连着左右各两个，共 4 条关系），实验随机断掉一些边，让受影响的 agent 从没连过的候选里挑一个新伙伴。三个组的初始 agent、被断的边、候选集完全相同，唯一的变量是挑伙伴时它能看到什么——只看连接关系；连接关系加上专长画像和准确率；以及不做偏好判断、由系统随机重连的对照组。

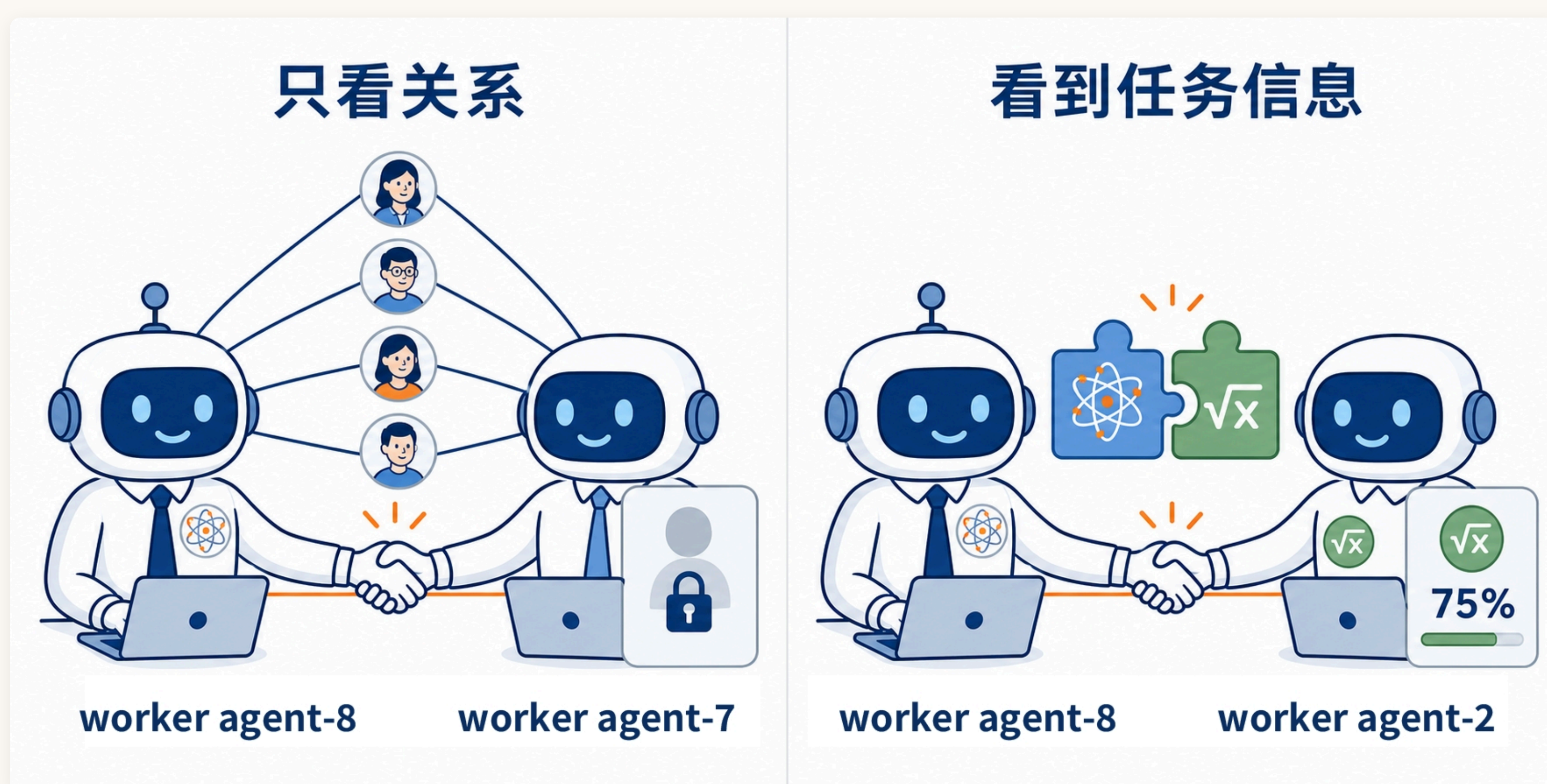


Figure 5：不同可见信息下的伙伴选择。只看社交关系时，agent-8 挑了有四个共同连接的"熟人"agent-7（准确率 0.25，但它当时看不到这个数）；能看到专长画像和准确率之后，同一个 agent-8 转而去找了没有共同邻居、但准确率 0.75、专长互补的 agent-2

原文用统计模型给每一种择友倾向拟了个系数，数越大表示"越倾向于这么挑"，接近 0 就是"基本不看这个因素"：

- 只给社交信息时，"找朋友的朋友"是压倒性的最强偏好，系数 +2.28——它能看到的只有关系网，那就只能按关系网挑。

- 一旦把专长画像和准确率摆上台面，同一个偏好掉到 +0.19，等于消失；接替它的是"找准确率高的" (+1.88) 和"找专长互补的" (+1.47)。

网络的形状也跟着变了。衡量"抱团程度"有个指标叫聚类系数——你的朋友之间彼此也是朋友的比例越高，这个数越大，直观上就是"小圈子有多密"。只看社交信息时它是 0.53（明显在抱团）；能看到任务信息之后掉到 0.28，而完全随机乱连的对照组是 0.27。

注意原文的结论不是"组织是涌现的"，恰恰相反——组织不会凭空出现；你让 agent 看到什么信息，就决定了这个系统奖励什么样的连接。 原文还有一句更硬的：自由分工不等于没有规则，你越希望 agent 自主，底层协议就越得可靠。

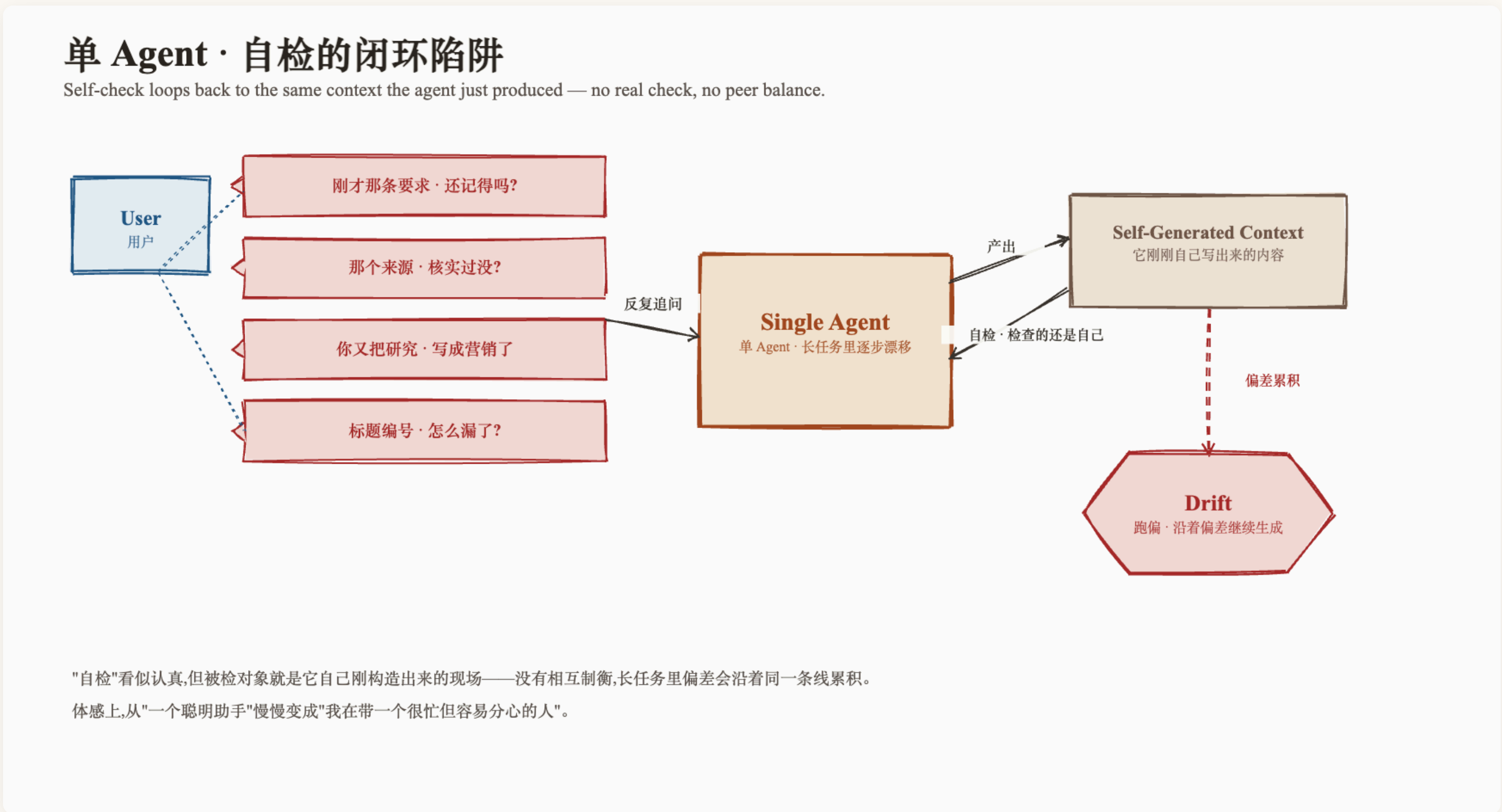
● MiniMax-单 agent 的毛病，与一个不是模型的 Team Engine

《MiniMax Agent Team: Built for Long-Running Tasks and Continuous Evolution》
(2026.5.27) 重点不在"我们做了 Agent Team"，而在"为此付了什么代价"。它先列单 agent 的几个毛病：

一是在你没预期的时刻停下。派一堆事，做完三件就开始汇报："我已经完成了 1、2、3，要不要继续？"原文把这归因于模型普遍存在的"上下文焦虑"——怕自己撑不到最后，索性早点交差。

二是越跑越傻。原文的说法是，用户会感觉它从"一个聪明助手"慢慢变成"我在带一个很忙但容易分心的人"。原因也简单：只要有一步跑偏了，后面所有步骤都会顺着这个偏差继续往下生成。

三是长任务跑起来之后一直没有回音。注意原文抱怨的不是"任务跑得久"——它整篇的立场恰恰是长任务本来就该跑很久——而是连一句"收到"都等不到：用户在 IM 里发完消息，是按"几秒内该有反应"来期待的，哪怕任务复杂，也想先听到一句"收到，我打算这么干，干完回来找你"，而不是盯着对话框十分钟、半小时，连任务有没有开始都不确定。"我的 Agent 怎么不回我"，是他们收到最多的一类用户反馈。



MiniMax 原文配图：单 agent 的自检是个闭环——它产出内容，然后回头检查自己刚造出来的现场，没有相互制衡，偏差就沿着同一条线一路累积下去

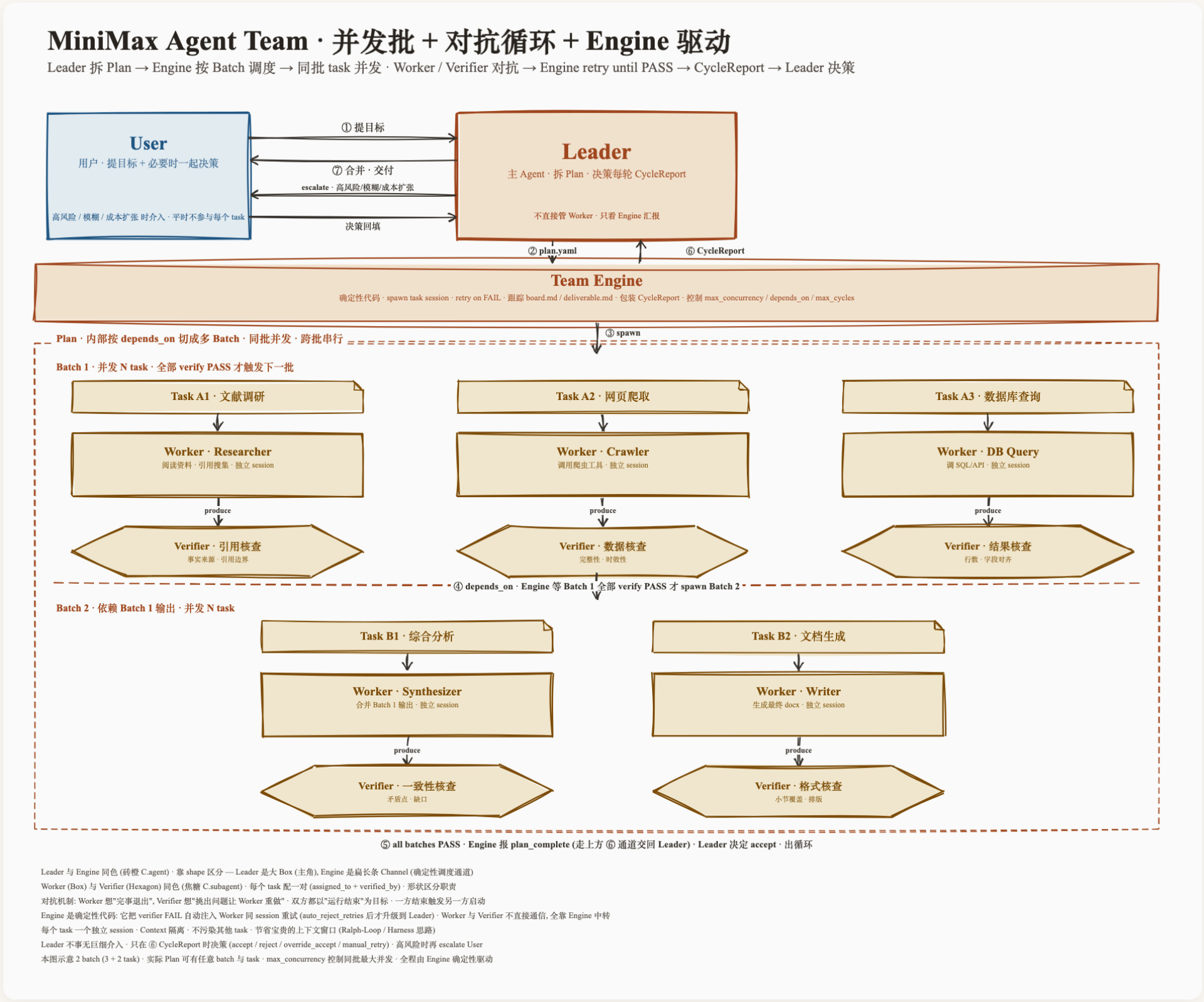
单 agent 在产出最终结果的时候，**不可避免地既是运动员又是裁判**。它可能很诚实地做了一遍自检，但它检查的那个对象，就是它自己刚刚造出来的现场。原文的措辞是单 agent 很难形成天然的

相互制衡。

三个角色，外加一个不是模型的 Engine

三个角色本身不新鲜：Leader 把用户目标翻译成任务结构，Worker 干活，Verifier 验收。Worker 和 Verifier 是明确的对抗关系，原文的比喻是公司里的研发和 QA——两边都想把活干完，但一边干完，恰恰是另一边开工的信号。

真正的设计重心，在中间那个 Team Engine。先说它不是什么：它不是又一个 agent，不是"再叫一个模型来当调度员"。它是一段普通的、写死的代码——同样的输入永远给出同样的结果，不会像模型那样这轮这么判、下轮那么判。



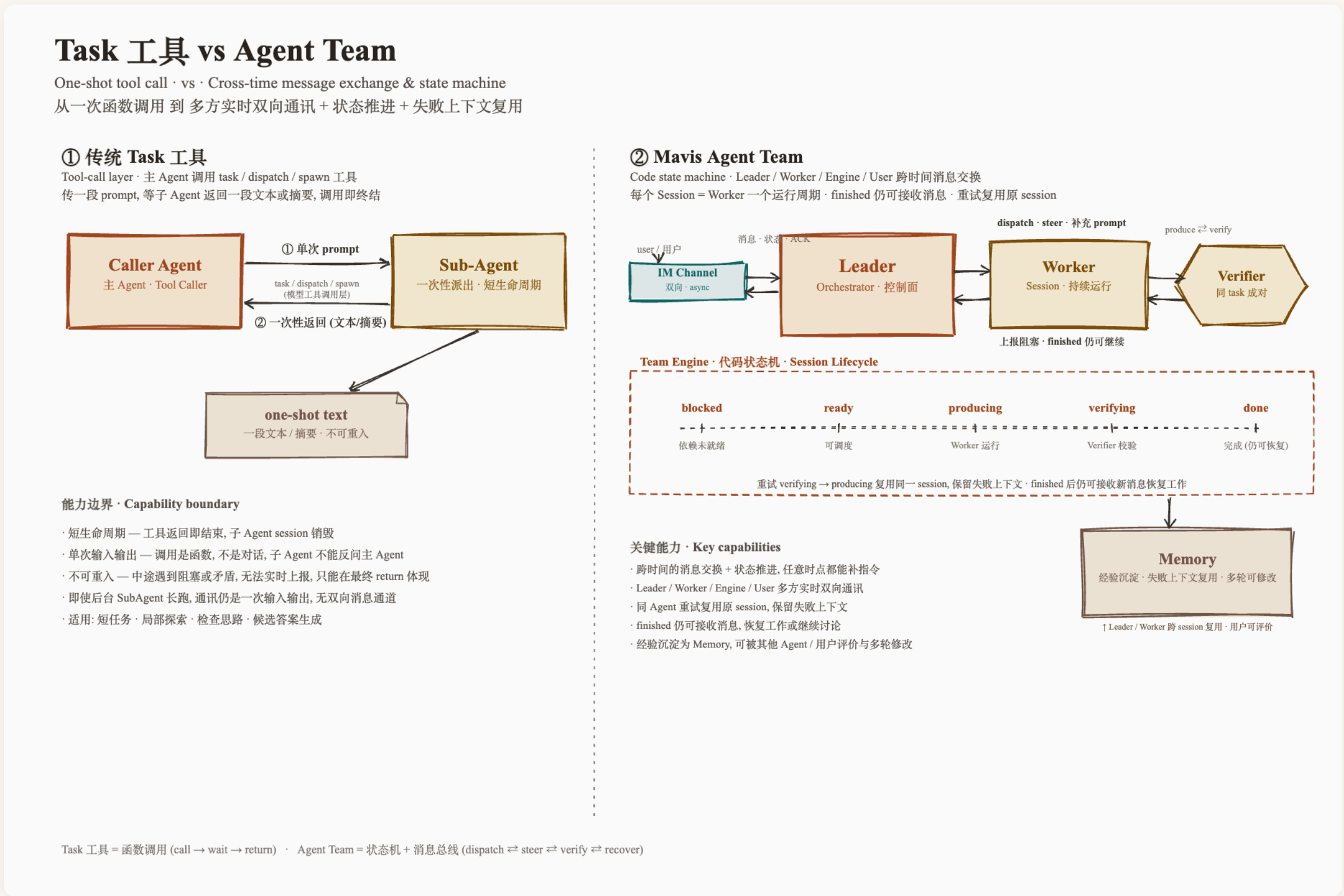
MiniMax 原文的全景图：Leader 拆出 Plan 交给 Team Engine，Engine 按 batch 调度、同批 task 并发跑，每个 Worker 配一个 Verifier 对抗，retry 到 PASS 才推进下一批，最后打包成 CycleReport 回给 Leader 决策

这张图里有几个正文没展开、但很能说明设计取向的细节：Plan 内部按 depends_on 切成多个 batch，同批并发、跨批串行；Engine 控制 max_concurrency（同批最大并发）和 max_cycles（最多重试几轮）；Leader 平时不介入每个 task，只在每轮 CycleReport 上做 accept / reject / override_accept / manual_retry，高风险的才 escalate 给人。

任务在 Engine 里走 producing → verifying → done 三个状态（图上那条更完整的链路还带了前置的 blocked / ready），verify 失败时 Engine 把 producing 节点重新唤醒继续改。

正文里另外补了一句，跟图上"Leader 不事无巨细介入"可以合起来读：Leader 全程能拿到 Engine 的最新状态，可以主动确认任务细节，甚至给正在跑的 Worker 或 Verifier 补一段 prompt；而在合并代码、覆盖生产数据这类高风险动作前面，最终判断必须由人签字。

它还专门画了一张图，对比"传统 Task 工具"和这套 Agent Team 的区别：



左边是传统的 task / dispatch / spawn 工具：传一段 prompt 进去，等一段文本出来，调用即终结——子 agent 不能反问主 agent，中途遇到阻塞或矛盾也没法实时上报，只能在最终 return 里体现。右边是状态机 + 消息通道：**blocked** → **ready** → **producing** → **verifying** → **done**，而且 finished 之后仍然能接收新消息、恢复工作

左边那一栏的四条"能力边界"，其实就是前面那个"有损压缩器"的机制层解释：问题不只是返回值被压缩了，而是发现不对的那一刻，信息根本没有通道传回去——等它写进 return 的时候，已经是事后总结了。

原文有一句总结这套东西的话很到位：多 Agent 系统是一个运行时（runtime），而不是把提示词编排一下。

• 多 agent 的三种成本

原文把多 agent 的额外开销拆成三块：

- 交接成本：同一份信息要在 agent 之间重新组织一遍。它的做法是落成文件——可读的交接文件，加一块跨 agent 的共享记事板，Worker 之间通过"文件路径 + 摘要"慢速通信，而不是把东西一次性塞进上下文。补一句：慢速的文件通道之外，原文还留了一条 agent 间通信的 CLI，agent 可以直接跟其他正在跑的节点说话——所以 MiniMax 这套并不是纯星型，Worker 之间是有直连能力的，只是默认走那条慢的。
- 共享成本：让每个 agent 都看到全部信息是要付钱的——每多共享一段内容，每个 Worker 的每一轮都要为它多烧一遍 token。
- 汇总成本：这条是原文说得最狠的一句——并行收集 10 份材料很容易，把它们合成一篇事实一致、引用对得上、风格统一的文章很难。承认这件事很贵，是设计一个 Team 的第一步。

第三条正是 Evomap 里提到的sub-agent形态的弊端。原文还强调这一步 Leader 的活是"把 10 份并成 1 份"，而不是"再叫几个人多产一点材料"。

它还引了一篇叫 Cost of Consensus 的研究，结论是：多 agent 的 token 消耗能到基线的 2.1—3.4 倍，准确率却没涨，有时还更差。

但这个数有前提：

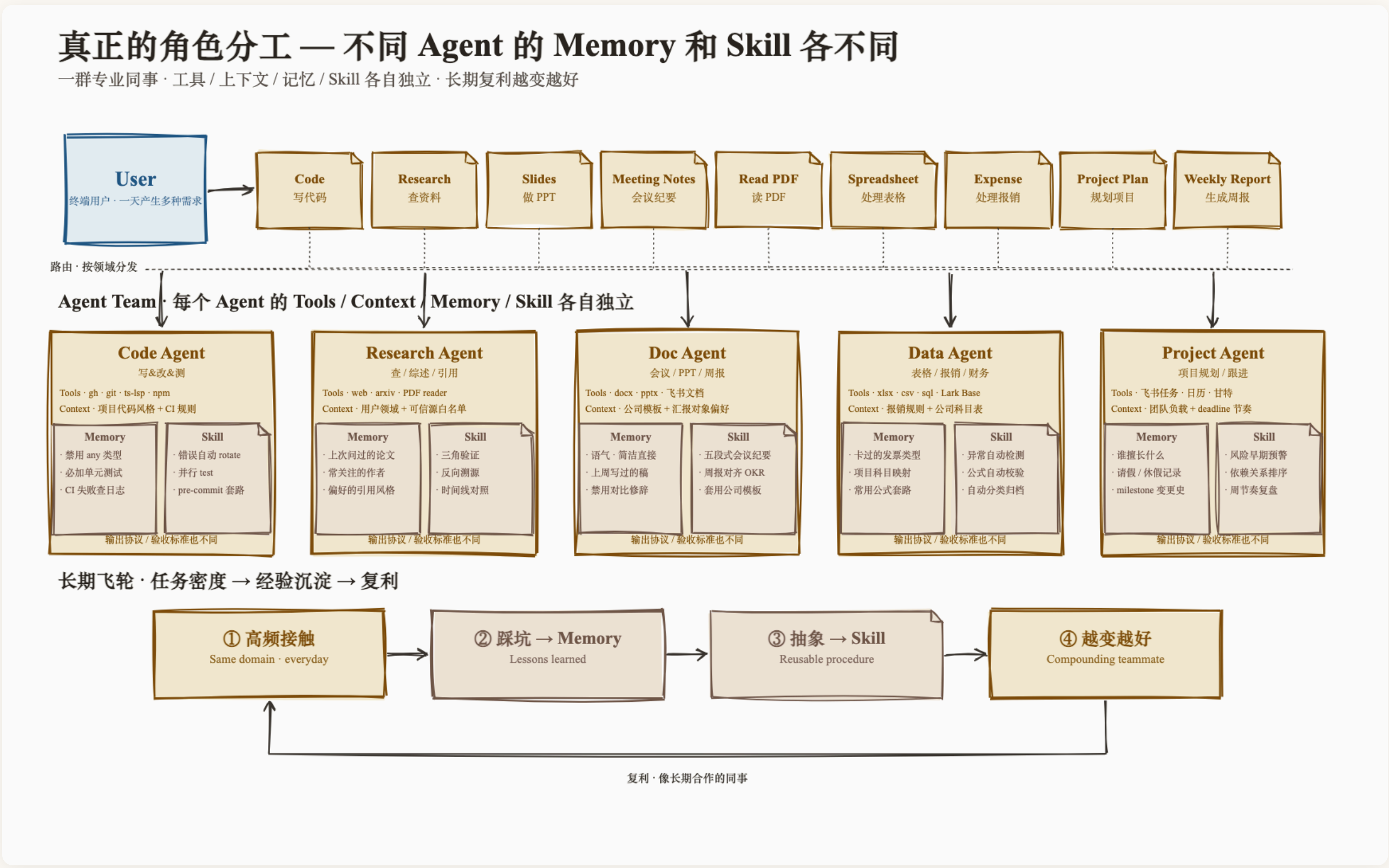
- "同质化辩论"指的是最朴素的那种多 agent 玩法——开几个一模一样的模型，给同一个问题，让它们互相看对方答案、来回辩论直到达成一致。大家能力相同、视角相同，辩到最后往往只是彼

此附和，多烧的 token 换不来新信息。

- "独立自我纠错"是拿来对比的基线——一个 agent 自己答完、自己回头检查一遍、自己改。

所以这个 2.1—3.4 倍量的是"多开几个一样的 agent 互相吵架"有多不划算，它并没有证明"角色分工明确、有验收、有停止条件的多 agent"不划算——而后者恰恰是 MiniMax 这套要做的事。原文的评语是：没有结构、没有停止条件的"更多"，只是把不确定性并行地摊开而已。

- 角色专业化不是换个 prompt



MiniMax 原文配图：Code / Research / Doc / Data / Project 各是一个 agent，每个的 Tools、Context、Memory、Skill 都各自独立；底下那条飞轮是"高频接触 → 踩坑写进 Memory → 抽象成 Skill → 越变越好"

靠 Skill 让单 agent 临时扮演不同角色，跟真正的角色专业化不是一回事。后者至少有四个维度——不同的工具、不同的上下文、不同的记忆、不同的 skill，而且从结果侧看，输出协议和验收标准也不一样。

文末给了一条判断标准，可以拿来回看前面几家、也用来看后面几家：

判断一个多 Agent 产品有没有价值，别看它能同时起多少个 Agent。看它能不能回答这几个问题：为什么拆、怎么验、什么时候停、失败怎么恢复、记忆怎么管。

原文还补了一句：这几个问题答得越清楚，它就越像一个生产系统；答得越含糊，就越像一个"演示用的群聊"。

- OpenAI Codex-只做了最轻的一层 subagent

个人觉得在agent这块openai做得相当保守。当claude code都在实验多种类型的agent协作时，openai 3月份才推出subagent，而且默认是不开启，只有当用户在指令里明确说请用subagent时，codex才会明确调用。

Codex 的 subagents 内置三种：explorer (read-heavy 的代码库探索)、worker (execution-focused，负责实现和修复)、default (兜底)。

内置三种，区别就在"准不准改你的代码"：`explorer` 只读不写，专门翻代码库、搞清楚现状；`worker` 是真动手的那个，负责写实现、改 bug；`default` 是兜底的万金油。

自定义 agent 就是一个个配置文件，放在 `~/.codex/agents/`（只对你自己生效）或项目里的 `.codex/agents/`（跟着仓库走）。每个必填三样：`name`（叫什么）、`description`（什么时候该用它）、`developer_instructions`（它的行为准则）；还能选填用哪个模型、思考多深、沙箱权限、挂哪些 MCP server、开哪些 skill。

拓扑是标准的星型——画出来就是一个中心点连着一圈外围点，所有子 agent 只跟中间那个主 agent 说话，彼此之间没有连线。而且是同步的：主 agent 把活派出去之后就在那儿等，等到最后一个子 agent 也交了活，才把所有结果并成一份回答返回。

有一点要说清楚：文档从头到尾只描述"主 agent 派活、主 agent 收活"，没给子 agent 之间提供任何互相说话的手段——但它也没有明文禁止，是只字未提。派活的触发也保守：在 ChatGPT Work 里，只有最高的 Ultra 智能等级会主动委派，其他等级都得你开口要；本地的 CLI 和 IDE 插件则是你直接要求、或者项目的 AGENTS.md / skill 指令写了要求，它才会派。

• Anthropic Claude Code-三种并存的多 agent 形态

Claude Code 是这几家里唯一把"多 agent"拆成三种并存产品形态的。

• Subagents：派出去，汇报回来

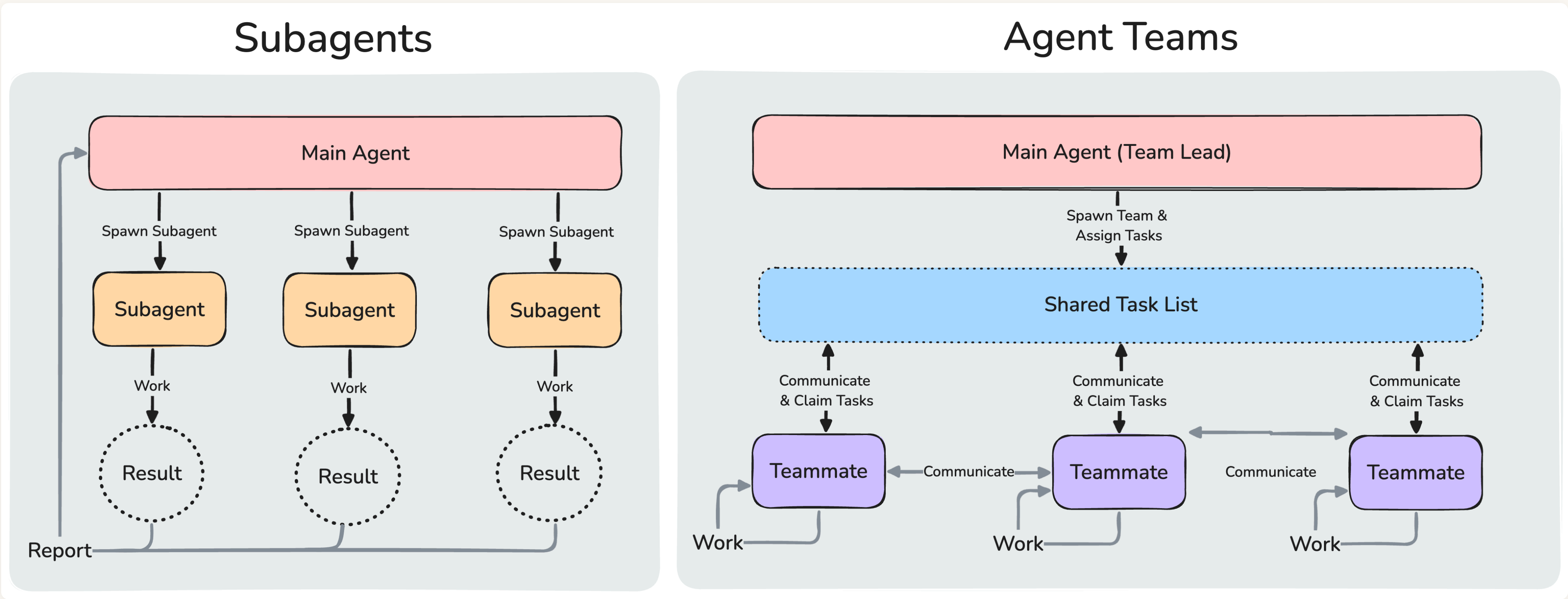
大家最熟悉的形态：主 session 把活派出去，子 agent 干完把结果汇报回来，子 agent 之间不说话。跟上一节 Codex 那套基本是同一个东西。

每个 subagent 有自己独立的上下文窗口，翻代码翻出来的几万行、试错试出来的一地报错，全都留在它自己那边，主 session 只收一句结论。

但也正因为如此，它有个绕不开的天花板——那句结论最后还是要落进 Claude 的上下文窗口里。派 3 个还好，派 200 个，主 session 一样会被撑爆。这就是后面 Dynamic Workflows 要解决的问题。

• Agent Teams：teammate 之间能直接说话

一个预览的试验性功能。



官方文档专门画了张图讲区别：左边 subagent 是主 agent 派出去、干完把 Result 汇报回来，彼此不说话；右边 agent team 是一张共享任务清单，teammate 自己认领、互相直接通信

机制上：

- 每个 teammate 都是一个完整独立的 Claude Code 会话——你可以理解成又开了一个跟你自己那个一模一样的 Claude Code，有自己独立的记忆空间，而不是主 agent 顺手派出去的一个小工具。它开机时会像正常启动一样把项目那套家当读一遍：项目说明文件（CLAUDE.md）、你挂的外部工具、你装的技能包。但有一件事它拿不到：你和 lead 之前聊的所有对话，它一个字都看不见。所以派活的时候，背景得在派活那句话里重新交代一遍，别指望它"你懂的"。用户可以绕过 lead 直接跟某个 teammate 对话——方向键选中、Enter 进去发消息、Esc 打断。
- 通信走 mailbox，实现是 `~/.claude/teams/{team-name}/inboxes/{agent-name}.json` 这样一个 JSON 文件，消息自动投递，lead 不用轮询。这里有个细节：`{team-name}` 不是你起的名字，是会话派生的——`session-` 加上 session ID 的前八位。
- 共享任务清单带依赖关系和文件锁：多个 teammate 同时抢一个任务时靠文件锁防竞态，前置任务完成后被阻塞的任务自动解锁。不过文档自己在限制那一节里补了一刀：任务状态会滞后——teammate 有时候干完了忘记把任务标成完成，结果就把依赖它的任务一直卡在那儿。
- 可以要求 teammate 先出计划再动手，lead 审批不通过就打回重写。而且这个审批是 lead 自主做的，不会来问你——文档说这是权限模型里唯一一个设计好的例外。
- 你还可以往三个关键时刻插一段自己写的小程序（官方叫 hook，钩子）当质量门。规矩很简单：这段程序跑完，如果以状态码 2 退出，就等于投了反对票，Claude Code 会拦下这个动作，并把你程序打印的话转达给对应的 agent。三个时刻的"反对"含义各不相同——插在 `TeammateIdle`（某个队友准备收工闲着了）上是"别下班，接着干，理由如下"；插在 `TaskCreated`（正要新建任务）上是"这个任务不许建"；插在 `TaskCompleted`（正要标记完成）上是"不算完，回去重做"。比如你可以写一句"测试没跑过就不许标完成"，然后它对每个队友都生效。

限制也不少：

- 默认是关着的。这还是个实验功能，得先设一个环境变量 `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1` 才打得开。

- 团队不能套娃。 teammate 不能再拉起一支自己的队伍，建队、招人永远只有 lead 说了算。（它倒是可以派自己的一次性 subagent，只是不能让那个 subagent 挂在后台跑。）
- 队长身份定死。 lead 永远是你最开始开的那个会话，中途不能把队长让给别人。
- 会话恢复救不回队友。 默认模式下（所有队友挤在同一个终端里跑）， `/resume`（恢复会话）和 `/rewind`（回退）都不会把队友一起恢复；更尴尬的是恢复之后，lead 可能还在继续给一群已经不存在的人发消息——这时候直接让它重新招一批就行。
- 一个会话只能带一支队。

官方建议是 3–5 个 teammate、每人 5–6 个任务，文档原话是：**三个专注的 teammate，常常胜过五个散的。**

• Dynamic Workflows：把计划挪进代码

《Orchestrate subagents at scale with dynamic workflows》做法是：Claude 现写一段 JavaScript 脚本，由一个独立的 runtime 在后台执行，用户会话保持可用。脚本里 `agent()` 派一个子 agent，`pipeline()` 对一个列表逐项派。官方原话是：workflow 把计划挪进了代码里。

而它后面紧跟的那句才是关键：

用 subagent、skill 和 agent team 的时候，Claude 本人就是那个编排者：它一轮一轮地决定下一步派谁、分什么活，而每一份结果最后都要落进某个上下文窗口。而 workflow 脚本自己攥着循环、分支和全部中间结果，所以 Claude 的上下文里只剩下最终答案。

中间结果落在脚本变量里，不落在 Claude 的上下文里。

举个具体的例子。假设你要扫 200 个文件，找出同一类 bug：

- 用 subagent：每个子 agent 查完一个文件，把结果汇报回主 session。200 份汇报全都要挤进 Claude 的上下文窗口，扫到一半就撑爆了，于是它只能边扫边压缩、边丢细节——Evomap 那 166 个丢掉的答案就是这么没的。
- 用 workflow：脚本里写一行 `const results = await pipeline(files, f => agent(...))`，那 200 份结果全都待在 `results` 这个 JS 变量里，Claude 从头到尾没看过它们。最后进 Claude 上下文的只有一句“发现 7 处，在这几个文件”。

差别不在于派了几个 agent，而在于**中间那 200 份东西有没有从模型脑子里过一遍**。这一下就把前面那个有损压缩器从主链路上摘掉了。

规模上：最多 16 个并发 agent（CPU 核少的机器会更少），单次运行总量上限 1,000 个。默认规模建议是 medium（15 个以内），可在 `/config` 改成 small（5 个以内）或 large（50 个以内）——注意这是“建议”不是硬上限，prompt 要求的规模不一样时会覆盖它。单次排到 25 个以上 agent、或预估 token 超 150 万，任务面板会挂一个 `Large workflow` 警告，但它纯属提示，不暂停也不限流。25 这个门槛还跟着设置走：你要是自己在 `/config` 里挑过档位，门槛就换成那个档位的数（挑了 small 就是 5），只有沿用内置默认才是 25。

内置的 `/deep-research` 是个现成的样本：扇出搜索、抓取、交叉核查、对每条 claim 投票，没通过交叉核查的 claim 直接从报告里滤掉。还有一条细节很讲究——verifier agent 因为限流或 API

报错检查不了某条 claim 时，报告会把它标成"未验证"，而不是当成"被推翻"。

跑完还能把这套流程存下来，下次一条命令复用。注意存下来的是什么——能复用的不是 worker 的定义，是那套编排本身，这是它跟前两套最大的区别。

• 官方对比表：谁持有计划

文档里这张表用的划分标准只有一个：计划握在谁手里。

	Subagents	Skills	Agent Teams	Workflows
它是什么	Claude 派出的一个 worker	Claude 遵循的一段指令	lead 监督一组平级 session	runtime 执行的一段脚本
谁决定下一步跑什么	Claude，逐轮决定	Claude，照着 prompt	lead agent，逐轮决定	脚本
中间结果存在哪	Claude 的上下文窗口	Claude 的上下文窗口	共享任务清单	脚本变量
可复用的是什么	worker 的定义	那段指令	team 的定义	编排本身
规模	每轮几个	同上	少量长跑的 peer	每次几十到几百个 agent
被打断之后	整轮重来	整轮重来	teammate 继续跑	同一会话内可续跑

表看着抽象，落到具体场景挺好分的：

- Subagents——一次性的脏活。 比如"把这个仓库里所有用到 `moment.js` 的地方找出来"。派三个出去分头翻，各自回一句话，主 session 汇总完就散了。它们之间不需要商量，你也不用中途插手。
- Skills——同一套动作反复做。 比如"每次写周报都按这个格式、先查这几个数据源、最后附一张表"。这压根不用多开 agent，它就是一份写死的操作规程，Claude 照着走。所以它的规模跟 subagent 一样，但复用的是那段指令本身。
- Agent Teams——需要互相抬杠的活。 官方举的例子是"应用收到一条消息就退出了，派 5 个 teammate 各查一个假设，让它们互相证伪"。关键在于它们得能互相说话——一个人挖到的线索要能当场推翻另一个人的结论。跨层改动也算：前端、后端、测试各归一个人，谁改完通知谁。
- Workflows——又多又重复，而且你不想让中间过程弄脏上下文。 官方给的例子是全仓库扫一遍 bug、500 个文件的迁移、一个需要交叉核查来源的研究问题。共同点是同一个动作要在几百个条目上各跑一遍，而你只想要最后那份结论。

一句话版的选法：成员之间要不要互相说话？要 → Agent Teams；不要但量很大 → Workflows；量不大、一次性 → Subagents；压根不用多开 agent、只是同一套流程反复走 → Skills。

• 真要上规模时，它放弃了团队形态

在 Anthropic 自己的文档里，Agent Teams 并不是它推荐用来上规模的那一套。那张四方表里，agent teams 的规模一栏写的是"少量长跑的 peer"，真正做几十到几百个 agent 的是 Workflows。

而 Workflows 的拓扑恰恰又回到了星型：agent 之间不直接说话，中间结果由脚本收拢。当 Anthropic 要把规模推上去的时候，它主动放弃了"团队"这个形态，换成了跟 Codex 更接近的结构——只不过把中间那个协调者从"模型逐轮判断"换成了"一段确定性的脚本"。

所以更准确的说法可能是：

- Codex 只做了一种形态，对应 Anthropic 表格里 subagents 那一列；验证、角色分工这些都留给用户自己用 TOML 配。
- Claude Code 做了三种，并且划了适用边界：文档给的二选一标准就一句话——你的 worker 之间需不需要互相说话。需要一批干活快、目标明确、干完汇报就走的，用 subagent；需要成员之间共享发现、互相质疑、自己协调的，用 agent team。
- "更像团队"和"更能扛规模"在当前的工程现实里是两件事，而且方向相反。团队形态的成本在于每个成员都是一个完整 session、通信是 $N \times N$ ；规模形态的前提则是你得先把任务拆成互不相干的原子单元。

• 两家同时画出一条红线

这一节最值得注意的，是两家在同一件事上给出了相同的警告。

Claude Code 的 Agent Teams 文档在讲适用场景时写道：

agent team 会带来协调开销，token 消耗也远高于单个会话。它最适合的场景，是 teammate 之间能各干各的。至于那些必须按顺序做的任务、要改同一个文件的任务、以及依赖关系很多的工作，用单个会话或者 subagent 反而更有效。

Codex 的文档在讲"什么活适合派子 agent"时写道：

一开始，先把并行 agent 用在读密集的活上——探索代码、跑测试、做分流、写摘要。写密集的并行流程要更小心，因为多个 agent 同时改代码会造成冲突，也会推高协调开销。

Anthropic 那篇 deep research 的工程博客也承认：

有些领域要求所有 agent 共享同一份上下文，或者 agent 之间存在大量依赖，这类场景在今天并不适合多 agent 系统。

同一份文件的编辑、依赖很多的工作——Claude Code 和 Codex 这两个包在模型外面的 agent 工具，都在自己的文档里劝退了。不过三家的力度不一样：Anthropic 那篇博客说得最重，直接讲这类场景"今天并不适合多 agent"；Codex 说的是"要更小心"；Claude Code 则是建议你退回单会话或 subagent。

那有没有人硬闯这条线？有。